Introduction to MicroC-OS
The Real Time Kernel

Session Speaker

Deepak V.

© M.S Ramaiah School of Advanced Studies - Bengaluru

Session Objectives

- To discuss the features of µcos-II
- To discuss Kernel Structure
- To discuss task management
- To study the APIs related to Time Management

Session Topics

- Kernel Structure
  - Tasks, Task States, Task Control Blocks, Task Scheduling
  - Interrupts under µcos-ii, Clock Tick
  - Initialization and starting of µcos-ii
- Task Management
  - Creating a Task
  - Deleting a Task
  - Suspending a Task
  - Resuming a Task
  - Getting Information about a Task
- Delaying a task
- Resuming a task
- System time get and set

© M.S Ramaiah School of Advanced Studies - Bengaluru

UC/OS-II Features

- Portable : UC/OS-II is written in highly portable ANSI C, with minimum microprocessor-specific code to make µcos-II easy to port on different processors. µcos-ii can run on most 8, 16, 32 or even 64-bit microprocessors or micro controllers and DSPs
- ROMable :UC/OS-II was designed for Embedded applications, proper tool chain (i.e, C compiler, assembler and linker/locator) is required to embed into products
- Scalable
- Preemptive
- Multitasking :µC/OS-II can manage up to 64 tasks
- Deterministic :Execution time for most of µC/OS-II functions and service are deterministic

© M.S Ramaiah School of Advanced Studies - Bengaluru

UC/OS-II Features

- Task Stacks : Each task requires its own stack size and hence reduces the amount of RAM needed for an application
- Services : Mailboxes, Queues, Semaphores, fixed-sized memory partitions, time-related functions
- Interrupt Management : Interrupt can be nested up to 255 levels deep
- Robust and Reliable
- **Note**
  - µC/OS-II does not support priority inheritance
  - With µC/OS-II, all tasks must have a unique priority
  - A task, also called a thread, is a simple program that thinks it has the CPU all to itself

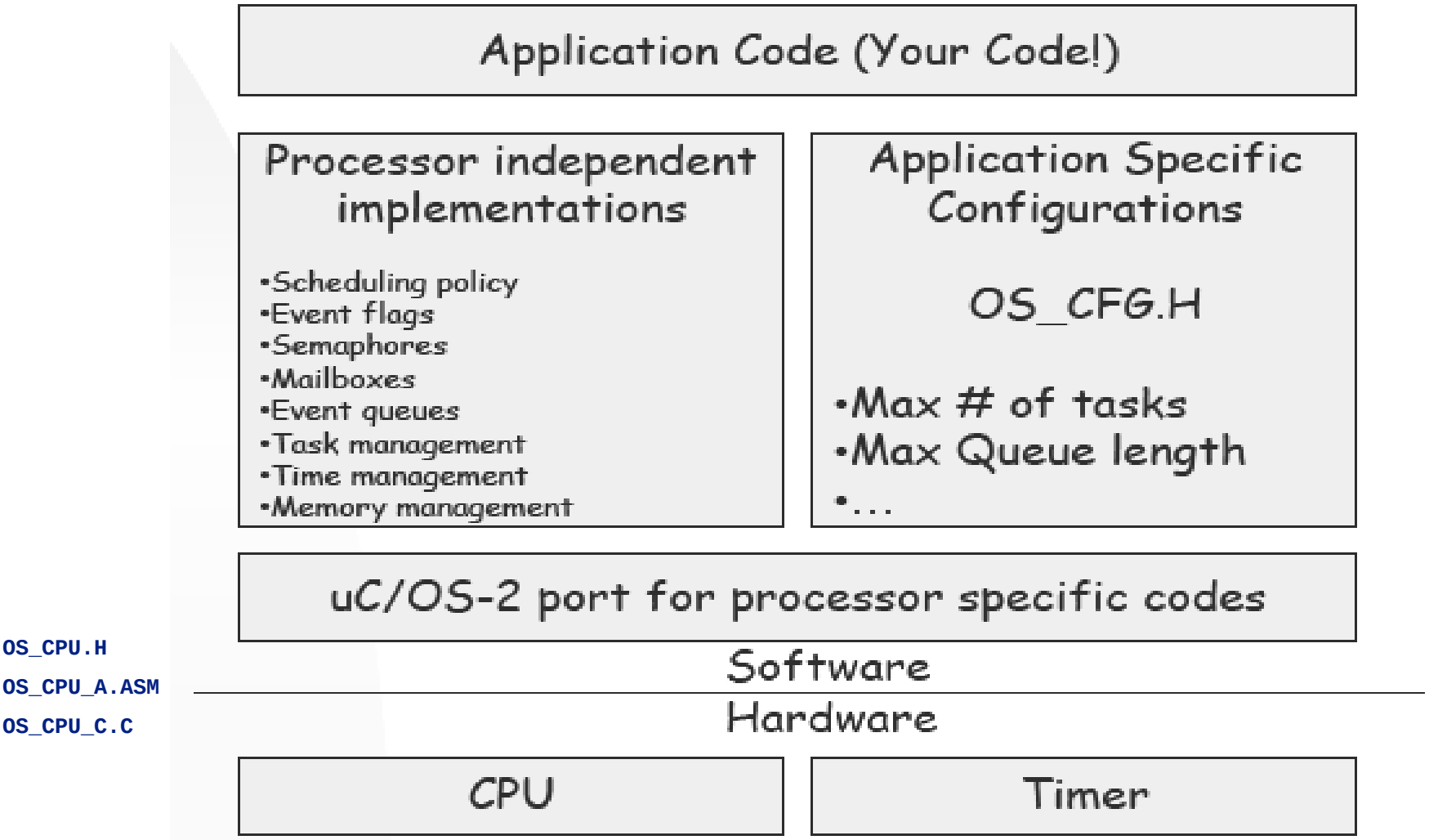
© M.S Ramaiah School of Advanced Studies - Bengaluru

Kernel Structure

- Task : Applications are often organized into independent cooperating programs
- Each of these programs while executing is called a task
- UC/OS-II can manage up to 64 such tasks
- The four highest priority tasks and the four lowest priority tasks are reserved for its own use and 56 application tasks
- The lower the value of the priority, the higher the priority of the task
- The task priority number also serves as the task identifier

© M.S Ramaiah School of Advanced Studies - Bengaluru

µC/OS-II File Structure



Application Services

- OS_ENTER_CRITICAL() and OS_EXIT_CRITICAL()
- OSInit()
- OSStart()
- OSIntEnter() and OSIntExit()
- OSSchedLock() and OSSchedUnlock()
- OSVersion()

© M.S Ramaiah School of Advanced Studies - Bengaluru

Critical Section( )

- A *critical section* is a portion of code that is not safe from race conditions because of the use of shared resources
- They can be protected by interrupt disabling/enabling interrupts or semaphores
  - The use of semaphores often imposes a more significant amount of overheads
  - A RTOS often use interrupts disabling/ enabling to protect critical sections
- Once interrupts are disabled, neither context switches nor any other ISR's can occur
- The two macros can be implemented using three different methods
- The method used is selected by the #define constant OS_CRITICAL_METHOD (defined in OS_CPU.H)

Critical Section( )

- μC/OS use two macro (find in OS_CPU.H) to disable and enable interrupts:
- OS_ENTER_CRITICAL();
- /* μC/OS critical section*/
- OS_EXIT_CRITICAL();

Tasks

- A task is an active entity which could do some computations
- Under real-time *u*C/OS-II systems, a task is typically an infinite loop function
  - Void Your (void *pdata)

```
{
 for(;;) {
/* USER CODE */
Call one of μC/OS's service:
/* USER CODE */
        }
}
```

- A task is created by OSTaskCreate() or OSTaskCreateExt()

Tasks

- *u*C/OS-II can have up to 64 priorities
  - Each task must be associated with an unique priority
  - 63 and 62 are reserved (idle, stat)
- The priority of a task can be changed by OSTaskChangePrio()
- A task could delete itself when it is done
- Actually you only have up to 56 tasks for use
- You can change OS_LOWEST_PRIO in OS_CFG.H make your tasks up to 62 tasks
- μC/OS can manage your task, by passing its address along with other argument to OSTaskCreate() or OSTaskCreateExt()
- In current μC/OS the task priority number also serves as task Identifier

© M.S Ramaiah School of Advanced Studies - Bengaluru

Tasks

```
void YourTask (void *pdata)                    (1)
{
        for (;;) {                             (2)
                /* USER CODE */
                Call one of uC/OS-II's services:
                OSMboxPend();
                OSQPend();
                OSSemPend();
                OSTaskDel(OS_PRIO_SELF);
                OSTaskSuspend(OS_PRIO_SELF);
                OSTimeDly();
                OSTimeDlyHMSM();
                /* USER CODE */
        }
}
```

© M.S Ramaiah School of Advanced Studies - Bengaluru

OSTaskDel()

A task that deletes itself when done:

```
void YourTask (void *pdata)
   {
         /* USER CODE */
         OSTaskDel(OS_PRIO_SELF);
   }
```

The priority of the current task

Task States

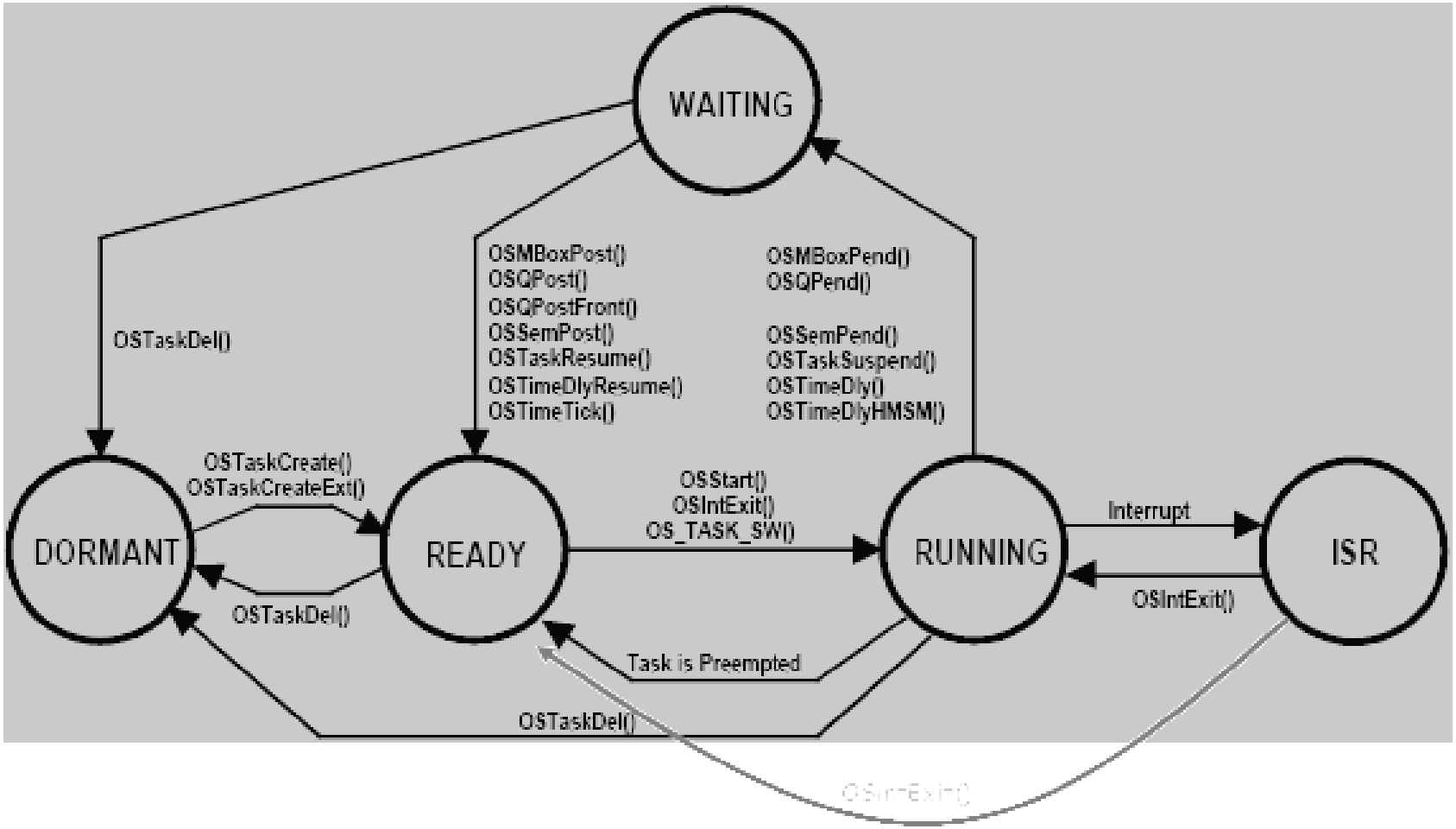
- **Dormant:** Procedures residing on RAM/ROM is not an task unless you call OSTaskCreate() to execute them
  - No tasks correspond to the codes yet!
- **Ready:** A task is neither delayed nor waiting for any event to occur
  - A task is ready once it is created
- **Running:** A ready task is scheduled to run on the CPU
  - There must be only one running task
  - The task running might be preempted and become ready

© M.S Ramaiah School of Advanced Studies - Bengaluru

Task States

- **Waiting:** A task is waiting for certain events to occur
  - Timer expiration, signaling of semaphores, messages in mailboxes, and etc.
- **ISR:** A task is preempted by an interrupt
  - The stack of the interrupted task is utilized by the ISR

Task States


WAITING
OSTaskDel()
OSMBoxPost()
OSQPost()
OSQPostFront()
OSSemPost()
OSTaskResume()
OSTimeDlyResume()
OSTimeTick()
OSMBoxPend()
OSQPend()
OSSemPend()
OSTaskSuspend()
OSTimeDly()
OSTimeDlyHMSM()
OSTaskCreate()
OSTaskCreateExt()
DORMANT
READY
OSStart()
OSIntExit()
OS_TASK_SW()
RUNNING
Interrupt
ISR
OSIntExit()
OSTaskDel()
Task is Preempted
OSTaskDel()
OSIntExit()

Task Control Block

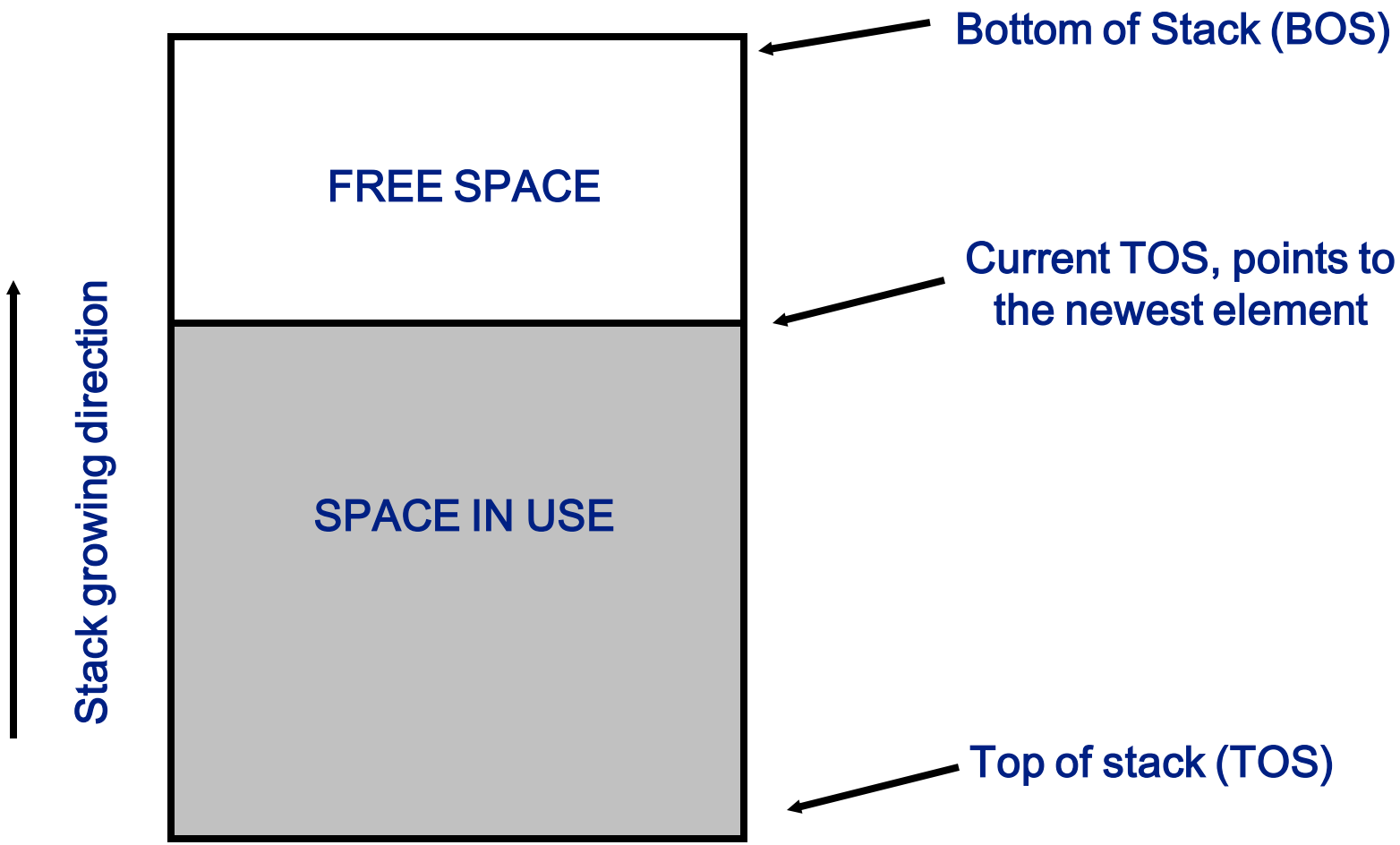
- When a task is created, it is assigned a Task Control Block OS_TCB
- OS_TCB is a data structure that is used by μC/OS-II to maintain the state of a task when it is preempted
- When task regain control of the CPU TCB allows task to resume execution exactly where it left off
- All OS_TCB reside in RAM
- An OS_TCB is initialized by the function *OS_Init()* when a task is created
- *OS_Init()* is called by either *OSTaskCreate()* or *OSTaskCreateExt()*

Task Control Block

- A TCB is a main-memory-resident data structure used to maintain the state of a task, especially when it is preempted
- Each task is associated with a TCB
  - All valid TCB's are doubly linked
  - Free TCB's are linked in a free list
- The contents of a TCB is saved/restored when a context- switch occurs
  - Task priority, delay counter, event to wait, the location of the stack
  - CPU registers are stored in the stack rather than in the TCB

© M.S Ramaiah School of Advanced Studies - Bengaluru

Task Control Block



Task Scheduling

- µcos-II always executes the highest priority task ready to run.
- The determination of which task has the highest priority, is determined by the scheduler
- Task level scheduling and ISR level Scheduling is performed by *OS_Sched()* and OSIntExit() function, respectively and found in the OS_CORE.C file
- All of the code in the *OS_Sched()* is considered a critical section
- Note that *OS_Sched()* could be written entirely in assembly language to reduce scheduling time
- *OS_Sched()* was written in C for readability and portability and to minimize use of assembly language

© M.S Ramaiah School of Advanced Studies - Bengaluru

Task Level Context Switch

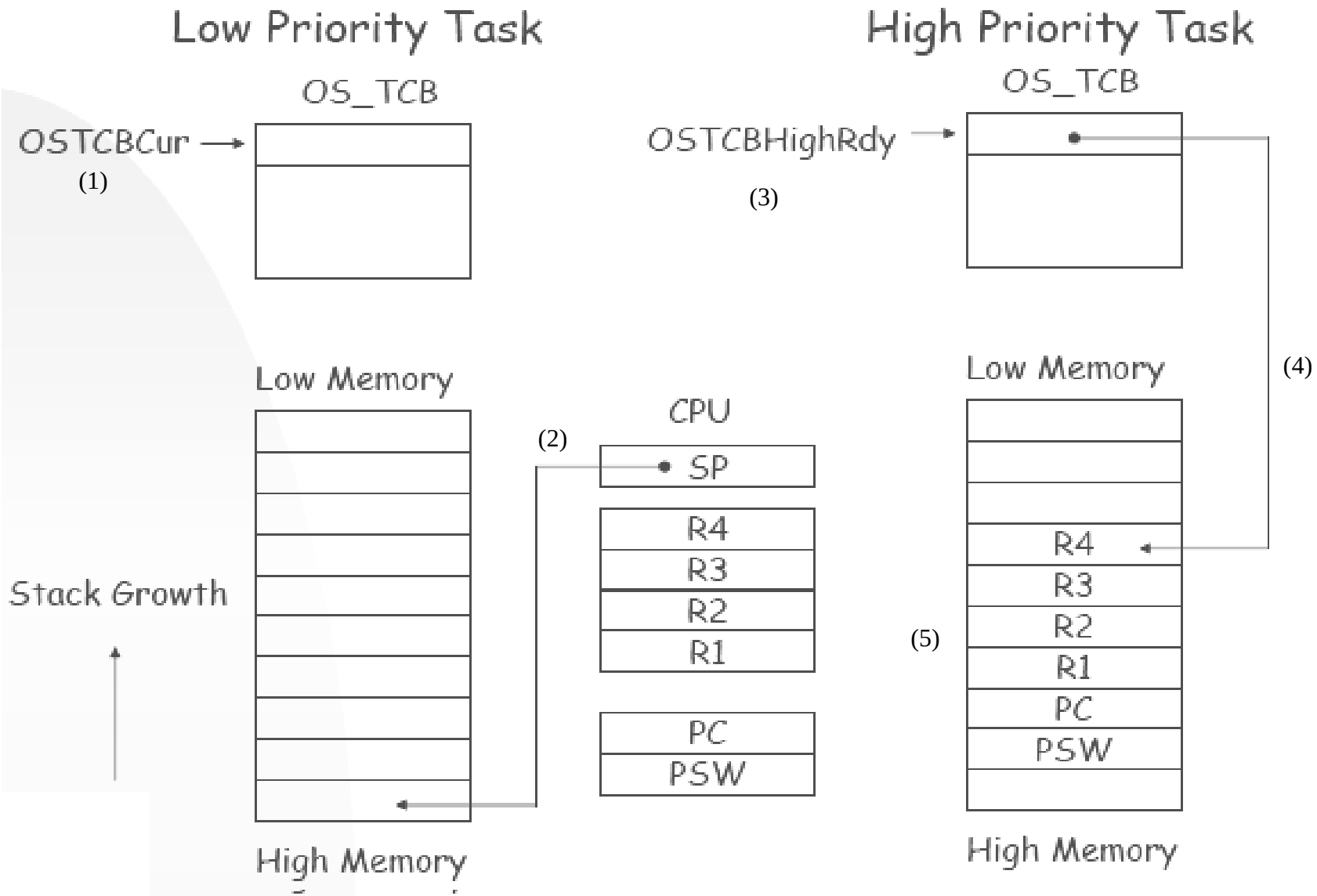
- A context switch must save all CPU registers and PSW of the preempted task onto its stack and then restore the CPU registers and PSW of the highest-priority ready task from its stack
- Task-level scheduling will emulate that as if preemption/scheduling is done in an ISR
  - OS_TASK_SW() will trigger a software interrupt
  - The interrupt is directed to the context switch handler OSCtxSw(), which is installed when *u*C/OS-II is initialized
- Interrupts are disabled during the locating of the highest- priority ready task to prevent another ISR's from making some tasks ready

Task Level Context Switch

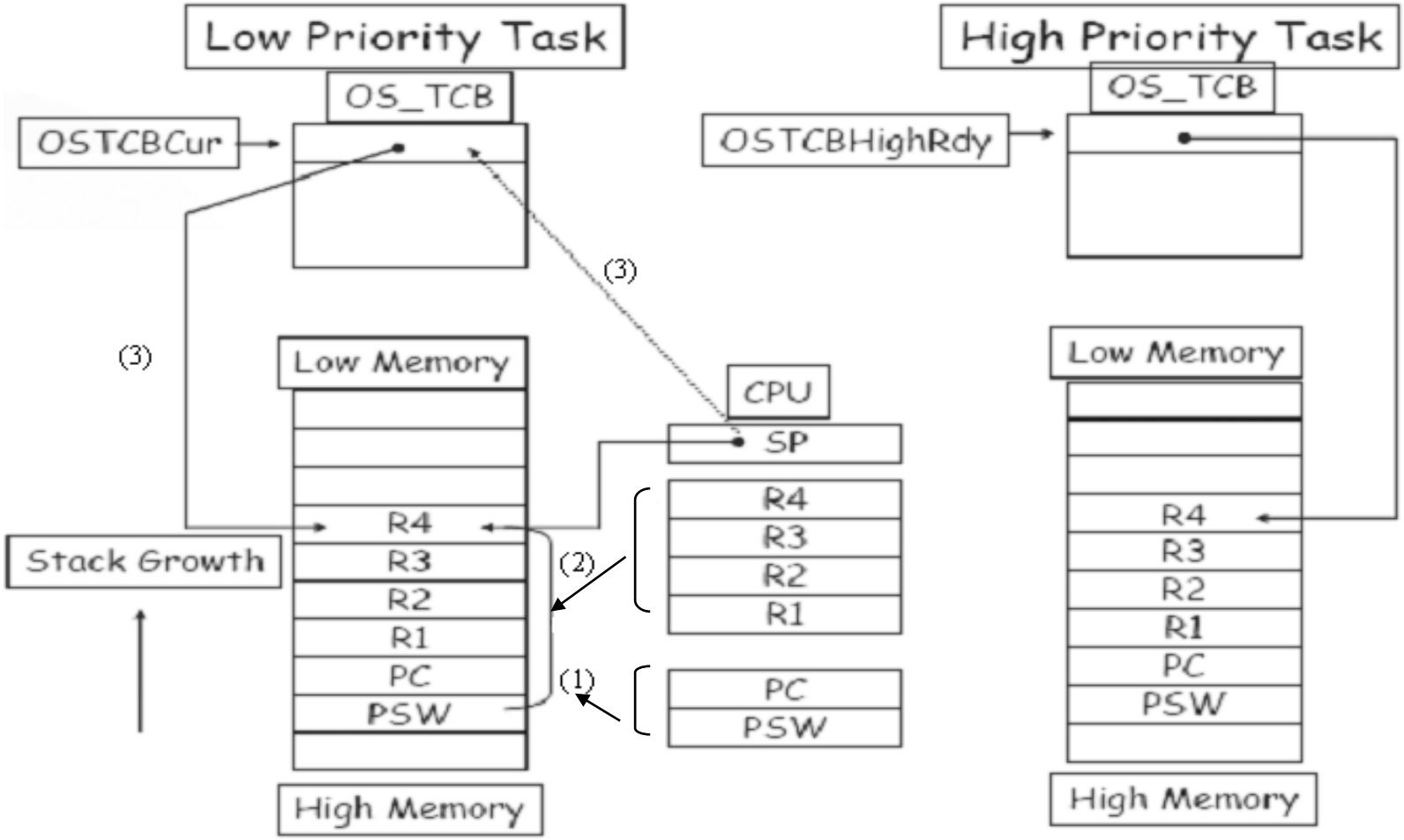
- After the scheduler has determined that a more important tasks needs to run, *OS_TASK_SW()* is called to perform a context switch
- *OS_TASK_SW()* suspends execution of the current task and allows the CPU to resume execution of the more important task
- The context of the task is generally the contents of the CPU registers
- The context-switch code simply needs to save the register values of the task being preempted and load into the CPU the values of the registers for the task to resume

© M.S Ramaiah School of Advanced Studies - Bengaluru

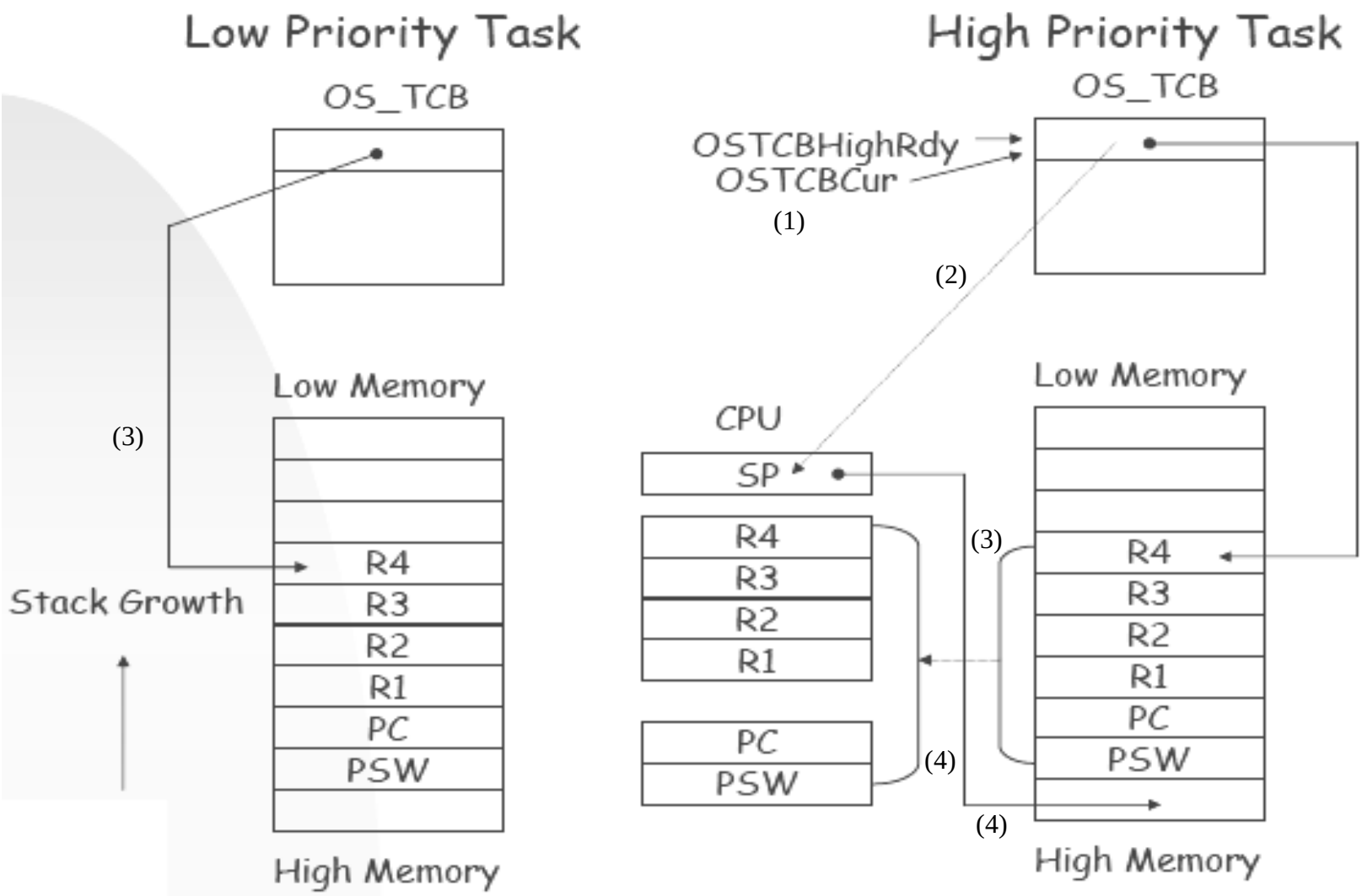
µC/OS-II structures when OS_TASK_SW( ) is called



Saving the Current Task's Context



Resuming the Current Task



Locking and Unlocking the Scheduler

- *OSSchedLock()* prevents high-priority ready tasks from being scheduled to run while interrupts are still recognized
- OSSchedLock() and OSSchedUnlock() must be used in pairs
- The task that call *OSSchedLock()* keeps control of the CPU even though other higher priority tasks are ready to run
- Even though the scheduler is locked interrupts will be still recognized and serviced
- The variable OSLockNesting keeps track of the number of times *OSSchedLock()* has been called

Idle Task

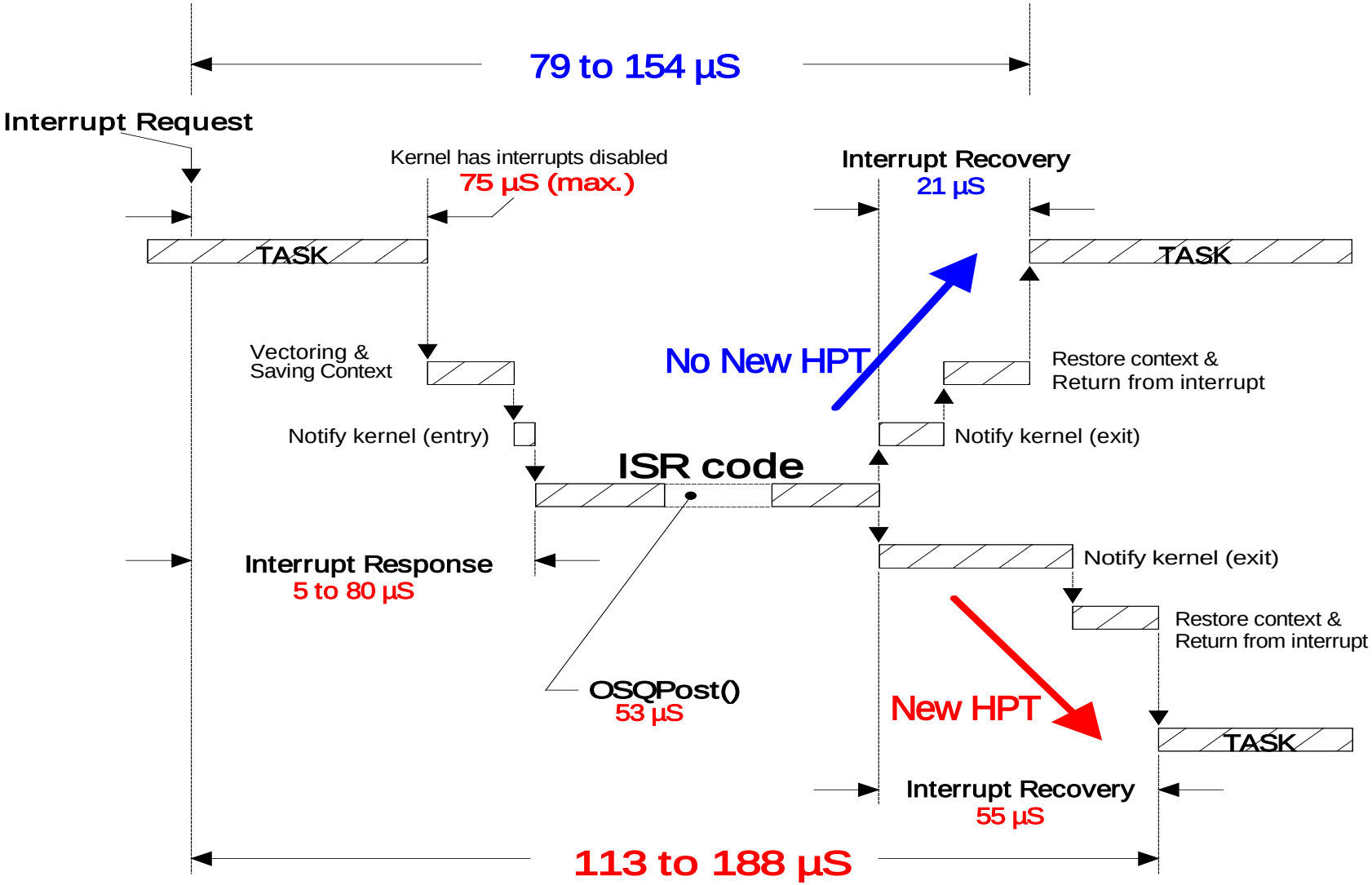
- µcos-II always creates a task that is executed when none of the other tasks are ready to run
- The idle task, *OS_TaskIdle()*, is always set to the lowest priority and cannot be deleted or suspended by application software
- To reduce power dissipation you can issue halt like instruction in the idle task

Statistics Task

- µcos-II contains a tasks that provides run-time statistics and it is called OS_TaskStat( )
- It is created by UC/OS-II. It executes every second to compute the percentage of the CPU usage
- The priority of this task is OS_LOWEST_PRIO – 1
- *OSStatInit()* must be called before *OSStart()* is called
- This value is placed in the signed 8-bit integer variable, OSCPUUsage
- The resolution of the OSCPUUsage is 1 percent

© M.S Ramaiah School of Advanced Studies - Bengaluru

Interrupts Under µcos-II



Clock Tick

- µcos-II requires a periodic time source to keep track of time delays and timeouts
- The actual frequency of the clock tick depends on the desired tick resolution of your application
- A tick should occur between 10 and 100 times per second or Hertz
- Two way to obtain a tick source:
  - Dedicating a hardware timer
  - Generating an interrupt from an AC power line signal
- Ticker interrupts must be enabled after multitasking has started
  - In the TaskStart() task of the examples
  - Do not do this before OSStart()
- Clock ticks are serviced by calling *OSTimeTick()* from a tick ISR
- Clock tick ISR is always a port (of UC/OS-II) of a CPU since we have to access CPU registers in the tick ISR

© M.S Ramaiah School of Advanced Studies - Bengaluru

Task

- A task is an active entity which could do some computations
- Under real-time µcos-II systems, a task is typically an infinite loop
- Each task must be associated with an unique priority
- µcos-II can manage up to 64 tasks and four highest priority tasks and four lowest priority tasks are reserved
- The application can make use of 56 tasks
- 63 and 62 are reserved (idle, statistics)
- A task is created by *OSTaskCreate*( ) or *OSTaskCreateExt*( )
- The priority of a task can be changed by *OSTaskChangePrio*( )
- A task could delete itself when it is done

© M.S Ramaiah School of Advanced Studies - Bengaluru

Creating a Task

OSTaskCreate()

INT8U *OSTaskCreate*(void (*task) (void *pd), void *pdata, OS_STK *ptos, INT8U prio);

- **Creates a task**
- **Tasks can be created either prior to the start of multitasking or by a running task**
- **A task cannot be created by an ISR**
- **A task must be written as an infinite loop, and must not return**
- **Related data structures are created according to the given parameters**
  - A Task Control Block (TCB)
  - The stack of the created task
  - The priority table
- **After a new task is created, the scheduler is called if multitasking is enabled**

Creating a Task

OSTaskCreateExt()

INT8U *OSTaskCreateExt*(void (*task)(void *pd), void *pdata, OS_STK *ptos, INT8U prio, INT16U id, OS_STK *pbos, INT32U stk_size, void *pext, INT16U opt);

- **Allows you to specify additional information about your task**
- **First four arguments are exactly the same**
- **Highly recommended because of its flexibility**

OSTaskCreateExt()

- task: a pointer-to-function points to the entry point of a task (note the syntax)
- pdata: a parameter passed to the task
- ptos: a pointer points to the top-of-stack
- prio: task priority
- id: task id, for future extension
- pbos: a pointer points to the bottom-of-stack
- stk_size: the stack size in the number of elements (OS_STK bytes each) 1 word under x86
- pext: an user-defined extension to the TCB
- opt: the options specified to create the task

Creating a Task[OSTaskCreate()]

- The priority of idle task is reserved by OSInit()
- Task's priority must unique priority
- μC/OS-II reserves the priority by placing a non_NULL pointer in OSTCBPrioTbl[]
- function OSTaskStkInit(task,pdata,ptos,0)responsible for setting up the task's stack
- OSTaskStkInit()return the new top_of _stack which saved in the task's OS_TCB
- OS_STACK_GROWTH reserves the stack grow
- call OSTCBInit()to obtain and initialize an OS_TCBfrom the pool of free OS_TCBs
- OSTCBInit(prio,psp,(void*)0,0,0,(void*)0,0) return OS_NO_ERR if task get tcb, OS_NO_MORE_TCB if no tcbin TCBFreeList

© M.S Ramaiah School of Advanced Studies - Bengaluru

Creating a Task[OSTaskCreate()]

OSTaskCreateExt() offers more flexibility but at the expense of additional overload.

```
INT8U OSTaskCreateExt(void (*task)(void *pd),
                      void *pdata,
                      OS_STK *ptos,
                      INT8U prio,
                      INT16U id,
                      OS_STK *pbos,
                      INT32U stk_size,
                      void *pext,
                      INT16U opt)
```

Task Priority

- Priority of a task must be a number between 0 and OS_LOWEST_PRIO
- Lower the number higher will be priority
- All tasks must have a unique priority

Task Options

- OS_TASK_OPT_STK_CHK
  - specifies whether stack checking is allowed for the task.
- OS_TASK_OPT_STK_CLR
  - specifies whether the stack needs to be cleared.
- OS_TASK_OPT_SAVE_FP
  - specifies whether floating-point registers are saved.

© M.S Ramaiah School of Advanced Studies - Bengaluru

Task Stacks

- A stack must be declared as being of type OS_STK and must consist of contiguous memory locations
- You can allocate stack space either statically or dynamically
- The stack must be a contiguous memory space
- static stack

```
Static OS_STK MyTaskStack[stack_size];
```

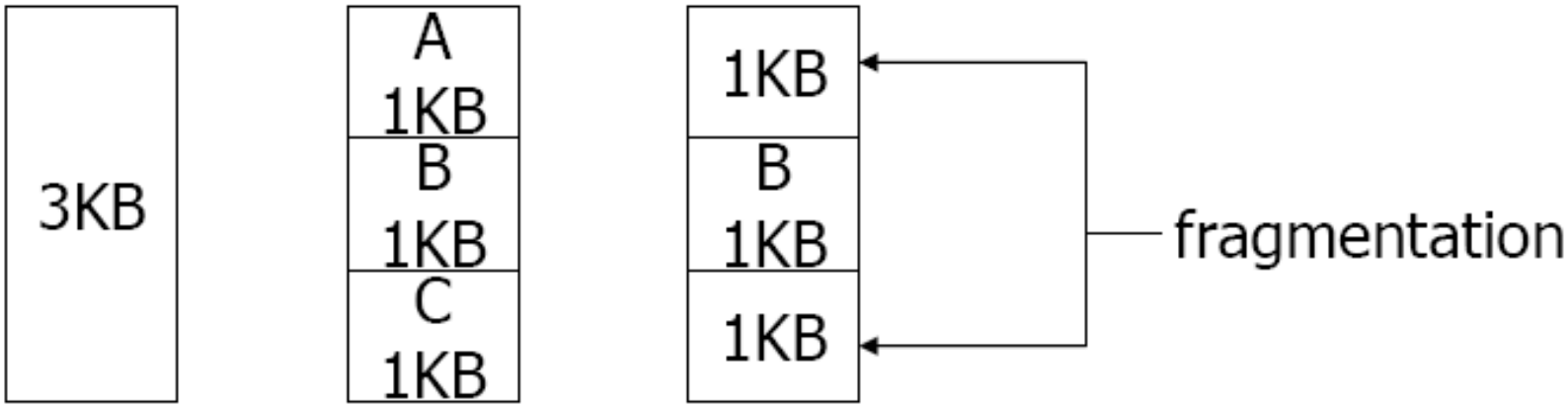
or

dynamically stack

```
OS_STK *pstk;
pstk=(OS_STK*)malloc(stack_size); /*C compiler's malloc() function*/
if(pstk!=(OS_STK*)0)
{
Create the task;
}
```

Task Stacks

Dynamically allocate must be careful with fragmentation



Task Stacks

- Stack grows either toward a high memory address or the low memory space
- Operations over stacks might not be byte-wise
  - The element size might not be 1 byte
    - 16 bits under x86
  - It is defined by a macro OS_STK
- Stack space can be statically declared variables or dynamically allocated space (malloc( ))
- Fragmentation might cause memory allocations (for stacks) failures
  - Fragmentation might not be a negligible issue because embedded systems adopt a small amount of RAM mostly
- Stacks might grow in different directions under different processors

© M.S Ramaiah School of Advanced Studies - Bengaluru

Task Stacks

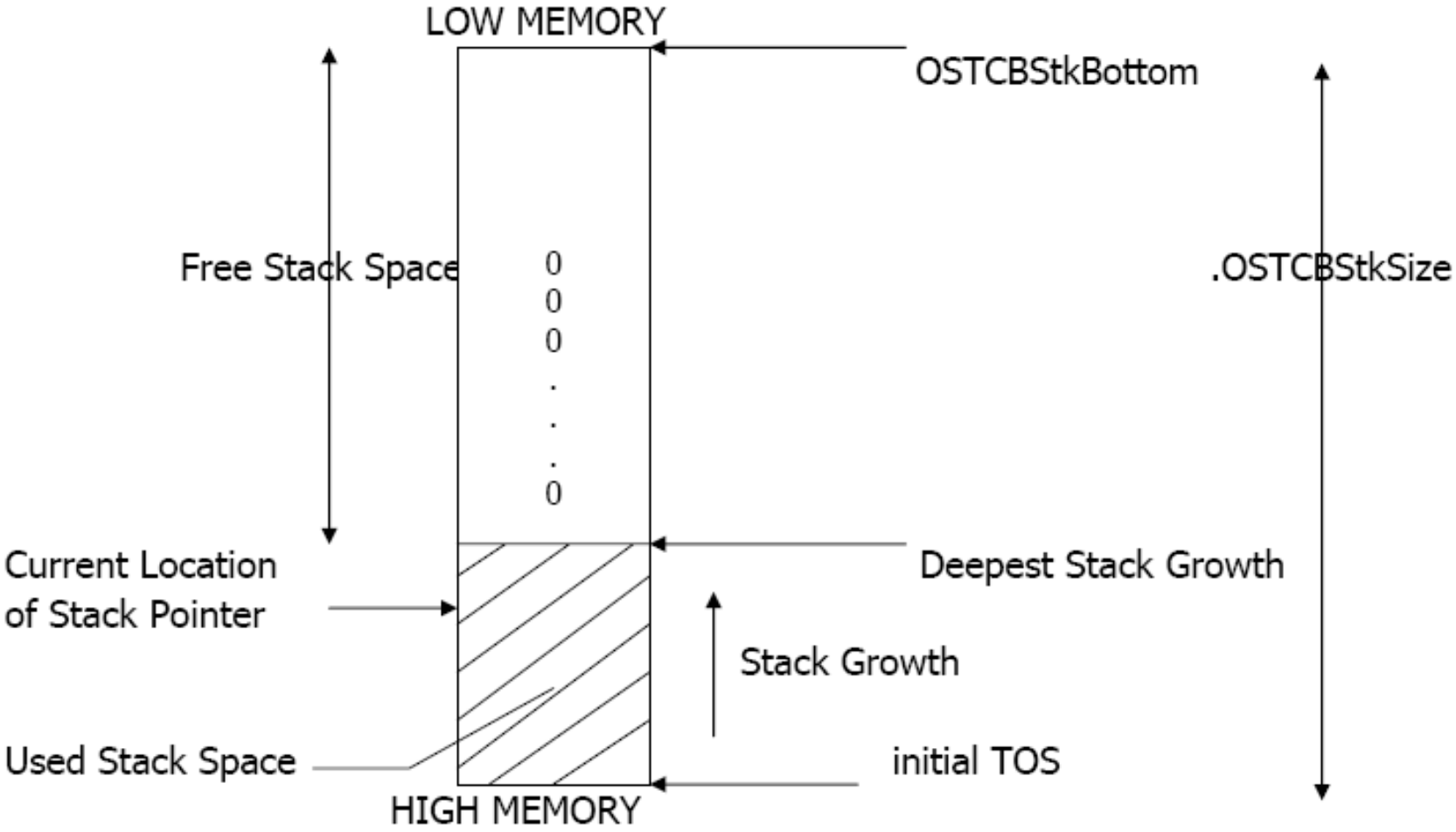
- It must be known how the stack grows when we call either OSTaskCreate() or OSTaskCreateExt() because we need to pass the task's top_of_stack to these functions

  OS_STK TaskStk[TASK_STK_SIZE];
- The size of task needed by your task is application specific

Stack Checking

- It is necessary to determine how much stack space a task actually uses
- Stack checking intends to determine the maximum run-time usage of stacks
- Stack checking allows to reduce the amount of RAM needed by the application code by not over-allocating the stack space
- µcos-II provides *OSTaskStkChk()*, which provides with the valuable information

Stack Checking



Stack Checking

- µC/OS-II determine stack growth by looking at the contents of the stack itself
- µC/OS-II requires that the stack be filled with zeros when the task is created
- If task created with OSTaskCreateExt(), the location of the bottom_of_stack and the size of the stack are stored in the task's OS_TCB
- OSTaskStkChk() computes the location of the bottom of stack and counting the number of zero-value entries on the stack until a nonzero value is found
- Used stack=total size-zero-value entries

© M.S Ramaiah School of Advanced Studies - Bengaluru

Deleting a Task

OSTaskDel()

INT8U *OSTaskDel*(INT8U prio);

- Deleting of a task means that the data structures (e.g., TCB) corresponding to the task-to-delete would be removed from main memory
  - The code resides on ROM are still there
- Deletes a task by specifying the priority number of the task to delete
- The calling task can be deleted by specifying its own priority number or OS_PRIO_SELF (if the task doesn't know its own priority number)
- The deleted task is returned to the dormant state
- Deleting of a task is slightly more complicated than creating since every resources/objects held by the task must be returned to the operating system
  - OSTaskDelReq()

© M.S Ramaiah School of Advanced Studies - Bengaluru

Requesting to Delete a Task

- Consider a task owns a resource such as memory buffer or a semaphore
- At this moment if any other task attempts to delete this task, the resource are not freed and thus are lost. Hence leads to memory leaks which is not accepted in any embedded system
- In this type of situation, tell the task that owns these resources to delete itself when it’s done with the resources by using an *OSTaskDelReq()*
- Both the requester and the task to be deleted need to call *OSTaskDelReq()*
- call OSTaskDelReq() to delete task which owns resources
- Both the requester and the task to be deleted need to call OSTaskReq()

Changing Priority of Task

OSTaskChangePrio()

INT8U *OSTaskChangePrio*(INT8U oldprio, INT8U newprio);

- Changes the priority of a task
- When you create a new task, you assign the task a priority
- At run time, you can change this priority dynamically by calling OSTaskChangePrio
- INT8U OSTaskChangePrio(INT8U oldprio, INT8U newprio)

© M.S Ramaiah School of Advanced Studies - Bengaluru

Suspending a Task

OSTaskSuspend()

INT8U *OSTaskSuspend*(INT8U prio);

- Suspends (or blocks) execution of a task unconditionally
- A suspended task can only resumed by calling the OSTaskResume() function call
- Calling task can be suspended by specifying its own priority number or OS_PRIO_SELF if the task doesn’t know its own priority number
- If a task being suspended is also waiting for time to expire, the suspension needs to be removed and the time needs to expire in order for the task ready to run
- INT8U OSTaskSuspend(INT8U prio)
- INT8U OSTaskResume(INT8U prio)

© M.S Ramaiah School of Advanced Studies - Bengaluru

Resuming a Task

OSTaskResume()

INT8U *OSTaskResume*(INT8U prio);

- Resumes a task suspended through the OSTaskSuspend()
- A suspended task can only resumed by calling OSTaskResume()
- Only function that can unsuspend a suspended task

Getting Information about a Task

OSTaskQuery()

INT8U *OSTaskQuery*(INT8U prio, OS_TCB *pdata);

- Obtains information about a task
- Application must allocate an OS_TCB data structure to receive a snapshot of the desired task's control block containing *every* field in the OS_TCB structure
- Be careful when accessing the contents of the OS_TCB structure, especially OSTCBNext and OSTCBPrev, because they point to the next and previous OS_TCBs in the chain of created tasks, respectively
- OSTaskQuery return a copy of the contents of the desired task's OS_TCB

© M.S Ramaiah School of Advanced Studies - Bengaluru

Resuming a Delayed Task

OSTimeDlyResume()

INT8U *OSTimeDlyResume*(INT8U prio);

- Resumes a task that has been delayed through a call to either OSTimeDly() or OSTimeDlyHMSM()

Initializing and Starting µcos-II

- OSInit() initializes data structures for *u*C/OS-II and creates OS_TaskIdle()
- OSStart() pops the CPU registers of the highest-priority ready task and then executes a return from interrupt instruction
- It never returns to the caller of OSStart() (i.e., main())

© M.S Ramaiah School of Advanced Studies - Bengaluru

Starting of *u*C/OS-II

```
void main (void)
{
OSInit(); /* Initialize uC/OS-II */
.
Create at least 1 task using either OSTaskCreate() or
OSTaskCreateExt();
.
OSStart(); /* Start multitasking! OSStart() will not return */
}
```

© M.S Ramaiah School of Advanced Studies - Bengaluru

Time Management

© M.S Ramaiah School of Advanced Studies - Bengaluru

Time Management Services

- OSTimeDly(),
  - Allow a task to delay itself for a number of ticks
- OSTimeDlyResume()
  - Resume a delayed task
- OSTimeGet(), OSTimeSet()
  - Set/obtain the system timer

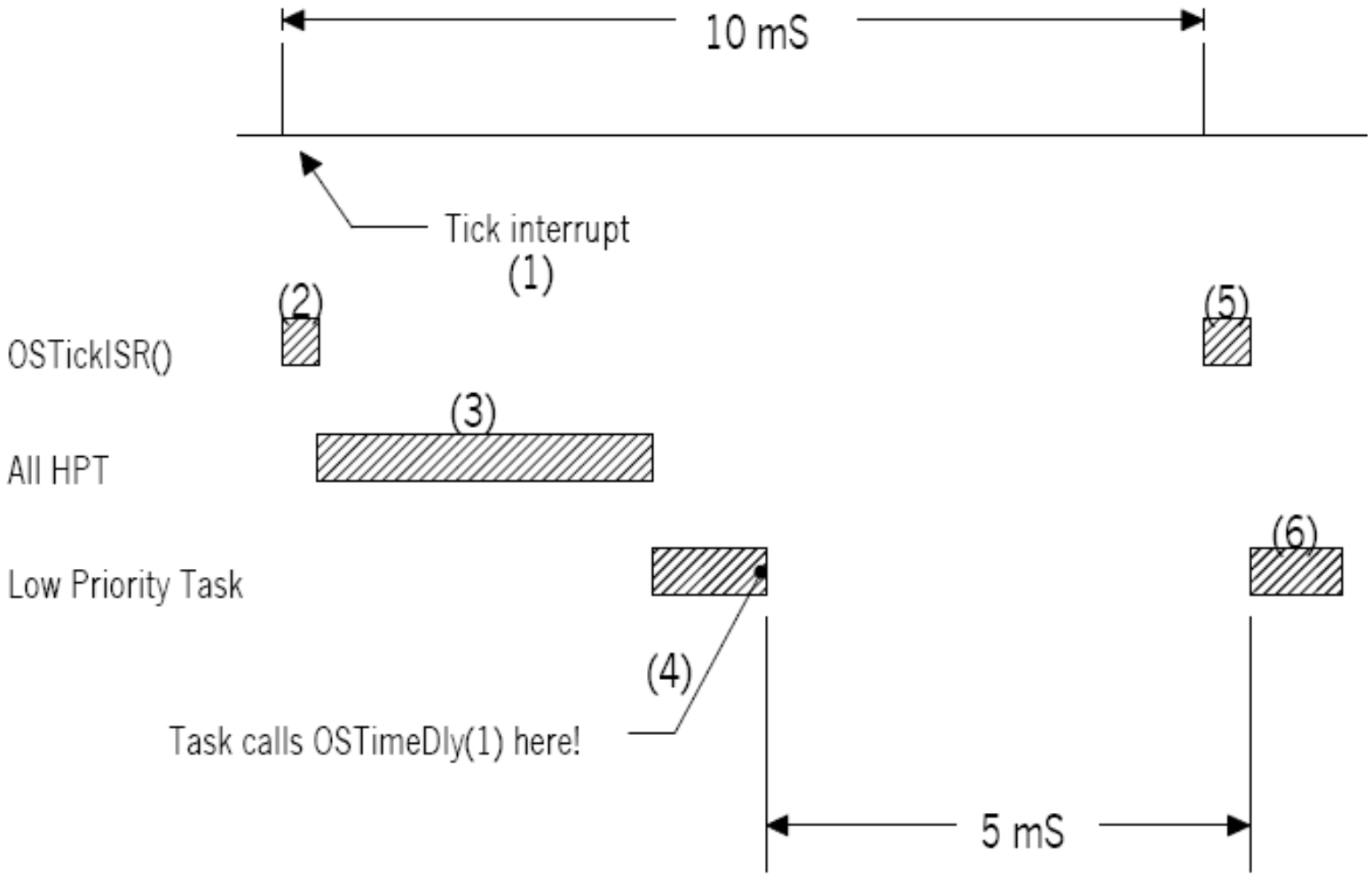
Delaying a Task

***OSTimeDly*()**

void *OSTimeDly* (INT16U ticks)

- Allows a task to delay itself for an integral number of clock ticks
- Rescheduling always occurs when the number of clock ticks is greater than zero
- Valid delays range from one to 65,535 ticks
- A delay of 0 means that the task is not delayed and *OSTimeDly()* returns immediately to the caller

Delay Resolution



Delaying a Task

***OSTimeDlyHMSM*( )**

void *OSTimeDlyHMSM*(INT8U hours, INT8U minutes, INT8U seconds, INT8U milli);

- Allows a task to delay itself for a user-specified amount of time specified in hours, minutes, seconds and milliseconds
- This format is more convenient and natural than ticks
- Rescheduling always occurs when at least one of the parameters is nonzero

© M.S Ramaiah School of Advanced Studies - Bengaluru

Resuming a Delayed Task

OSTimeDlyResume()

INT8U *OSTimeDlyResume*(INT8U prio);

- Resumes a task that has been delayed through a call to either *OSTimeDly()* or *OSTimeDlyHMSM()*

Get System Time

OSTimeGet()

INT32U *OSTimeGet*(void);

- Obtains the current value of the system clock
- The system clock is a 32-bit counter that counts the number of clock ticks since power was applied or since the system clock was last set

© M.S Ramaiah School of Advanced Studies - Bengaluru

Set System Time

OSTimeSet()

void *OSTimeSet*(INT32U ticks);

- Sets the system clock
- The system clock is a 32-bit counter that counts the number of clock ticks since power was applied or since the system clock was last set

Summary

- A task is an active entity which could do some computations
- Each task requires its own stack size and hence reduces the amount of RAM needed for an application
- A TCB is a main-memory-resident data structure used to maintain the state of a task, especially when it is preempted
- A *critical section* is a portion of code that is not safe from race conditions because of the use of shared resources
- A context switch must save all CPU registers and PSW of the preempted task onto its stack and then restore the CPU registers and PSW of the highest-priority ready task from its stack
- A suspended task can only resumed by calling the OSTaskResume() function call
- µcos-II requires a periodic time source to keep track of time delays and timeouts

© M.S Ramaiah School of Advanced Studies - Bengaluru